

ROS core components

ROBOTICS



POLITECNICO
MILANO 1863



Today schedule

- Using **classes** → how to properly write your code! ⇒ Best way when there are more nodes acting simultaneously
- **tf**
- **Dynamic reconfigure**
- **Message filter**
... basic ROS components ... → all necessary for (1st project) !
- **First project**
↓
then all specific components and advanced package for navigation ecc...

PUB/SUB code written on same node!



PUB/SUB in the same node (good practice)

ROBOTICS



POLITECNICO
MILANO 1863

PUB-SUB



Up to now a node was a publisher or a subscriber, if we want to do both tasks in the same node we will need a more elaborated structure: good practice is to create a class which contains both the publisher and the subscriber

PUB-SUB



To test our publisher-subscriber node we will subscriber to two different unsynchronized topics and re-publish them at constant rate using the last message received.

First we create a `test_pub.cpp` file which will publish two topics

on /pub_sub_same package on the /src folder of
/robotics workspace
✓ on that package /src
↓

test_pub.cpp

```
GNU nano 4.8 test_pub.cpp
#include "ros/ros.h"
#include "std_msgs/String.h"
#include <stdlib.h>

#include <sstream>

int main(int argc, char *argv[])
{
    ros::init(argc, argv, "publisher");
    ros::NodeHandle n;

    ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 10);
    ros::Publisher chatter_pub2 = n.advertise<std_msgs::String>("chatter2", 10);

    ros::Rate loop_rate(10);
    int count = 0;
```

Initialize publisher mode

} 2 publishers on that mode
on topic chatter / chatter2

string msg

while...

```
std_msgs::String msg;

std::stringstream ss;
ss << "hello world " << count;
msg.data = ss.str();

ROS_INFO("%s", msg.data.c_str());

if (rand() % 10 < 6){
    chatter_pub.publish(msg);
}
if (rand() % 10 < 2){
    chatter_pub2.publish(msg);
}
```

then we publish our data on random instant of time

just

two publishers

code as seen before...

test_sub.cpp

→ the subscriber

```
GNU nano 4.8 test_sub.cpp
#include "ros/ros.h"
#include "std_msgs/String.h"

class pub_sub
{
    std_msgs::String message1;
    std_msgs::String message2;

private:
    ros::NodeHandle n;

    ros::Subscriber sub1; } 2 subscribers
    ros::Subscriber sub2; } ← publisher
    ros::Publisher pub;

    // Read 56 lines
```

subscriber to both the topic from 1st 2nd publisher and re-publish the mode at same time

↓ mode with two sub on "chatter"/"chatter2" + publisher on that mode

↓ you have to do it using class!

ros::Timer timer1; ← to synchronize the data → using timer... publishing @ specific frequency!

```
public:
    pub_sub(){
        sub = n.subscribe("/chatter", 1, &pub_sub::callback, this); ← which topic subscribing
        sub2 = n.subscribe("/chatter2", 1, &pub_sub::callback2, this);
        pub = n.advertise<std_msgs::String>("/rechatter", 1); ← publisher and topic
        timer1 = n.createTimer(ros::Duration(1), &pub_sub::callback1, this);
    }
    void callback(const std_msgs::String::ConstPtr& msg){ ← callback to call (1st sub callback)
        message1=*msg; ← 1st msg
    }
}
```

```
void callback2(const std_msgs::String::ConstPtr& msg){ ← same for 2nd sub
    message2=*msg; ← received msg saved
}
```

```
void callback1(const ros::TimerEvent&){ ← callback from timer called @ 1Hz
{
    pub.publish(message1);
    pub.publish(message2);
    ROS_INFO("Callback 1 triggered");
}
```

```
int main(int argc, char **argv)
{
    ros::init(argc, argv, "subscribe_and_publish");
    pub_sub my_pub_sub;
    ros::spin();
    return 0;
}
```

on main we just init the mode... create the class and .spin();

all happens inside the class

↑ code handling multiple sub + publish data...

(it can be done without classes but waste resources!) ← BAD!

TESTING this... after `catkin-make` to compile all

>> `roscore`

>> `roslaunch pub-sub-same test-pub`
... start the publisher!

>> `rostopic list`

`chatter`
`chatter2` } the one created
by us...

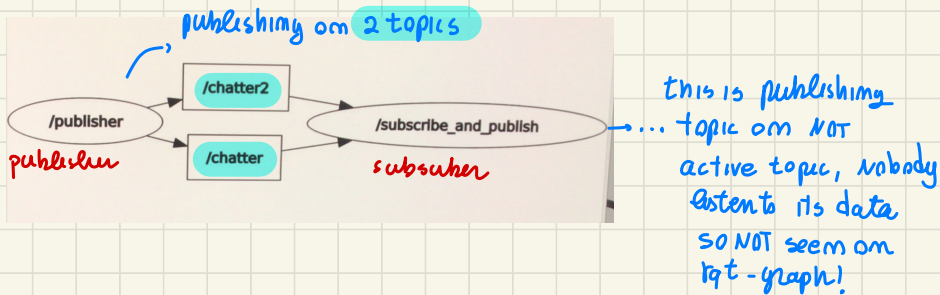
>> `roslaunch pub-sub-same test-sub`

↳ callback triggered

↳ publishing on /rechatter topic

receiving data from the 2 publishers

>> `roslaunch rqt-graph rqt-graph`



↑

IDEA on

how to structure

the code when more pub/sub!

PUB



```
#include "ros/ros.h"
#include "std_msgs/String.h"
#include <stdlib.h>
#include <sstream>
int main(int argc, char *argv[])
{
    ros::init(argc, argv, "publisher");
    ros::NodeHandle n;
    ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 1000);
    ros::Publisher chatter_pub2 = n.advertise<std_msgs::String>("chatter2", 1000);
    ros::Rate loop_rate(100);
    int count = 0;
```

← Standard ROS include
plus stdlib for random

↓ we create two publisher

PUB



```
while (ros::ok())
{
    std_msgs::String msg;
    std::stringstream ss;
    ss << "hello world " << count;
    msg.data = ss.str();
    ROS_INFO("%s", msg.data.c_str());
    if (rand() % 10 < 6) {
        chatter_pub.publish(msg);
    }
    if (rand() % 10 < 2) {
        chatter_pub2.publish (msg);
    }
}
```

← We randomly publish a message

PUB-SUB



```
int main(int argc, char **argv)
{
    ros::init(argc, argv, "subscribe_and_publish");
    pub_sub my_pub_sub;
    ros::spin();
    return 0;
}
```

In the main function we only initialize ROS, we don't create a NodeHandle, but we create the my_pub_sub object

PUB-SUB



```
class pub_sub
```



All the ros code will be inside this class

```
{
```

```
std_msgs::String messagio;
```



Here we save the message that we want to republish

```
std_msgs::String messagio2;
```

```
private:
```



NodeHandle, publisher and subscriber are created here as private

```
ros::NodeHandle n;
```

```
ros::Subscriber sub;
```

```
ros::Subscriber sub2;
```

```
ros::Publisher pub;
```

```
ros::Timer timer1;
```

PUB-SUB



public:

```
pub_sub() {  
    sub = n.subscribe("/chatter", 1, &pub_sub::callback, this);  
    sub2 = n.subscribe("/chatter2", 1, &pub_sub::callback2, this);  
    pub = n.advertise<std_msgs::String>("/rechatter", 1);  
    timer1 = n.createTimer(ros::Duration(1), &pub_sub::callback1, this);  
}
```

Here we set our two subscriber, the publisher and a timer at 1Hz

PUB-SUB



```
void callback(const std_msgs::String::ConstPtr& msg) {  
    messagio=*msg;  
}
```

↑ First message callback

```
void callback2(const std_msgs::String::ConstPtr& msg) {  
    messagio2=*msg;  
}
```

↑ Second message callback

```
void callback1(const ros::TimerEvent&)  
{  
    pub.publish(messagio);  
    pub.publish(messagio2);  
    ROS_INFO("Callback 1 triggered");  
}
```

← Timer callback which publish the two messages



ROBOTICS

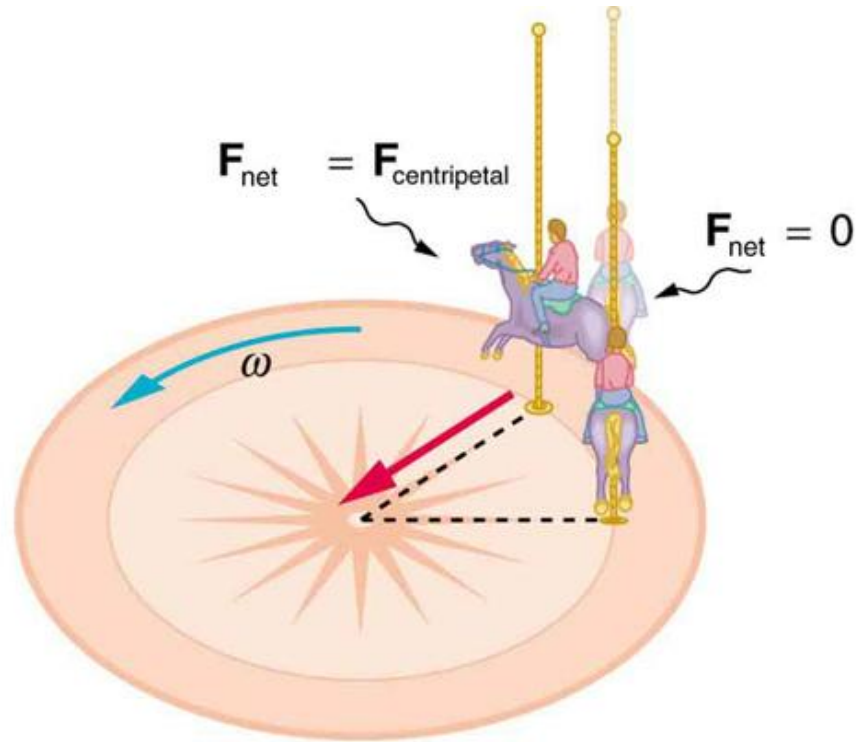


POLITECNICO
MILANO 1863

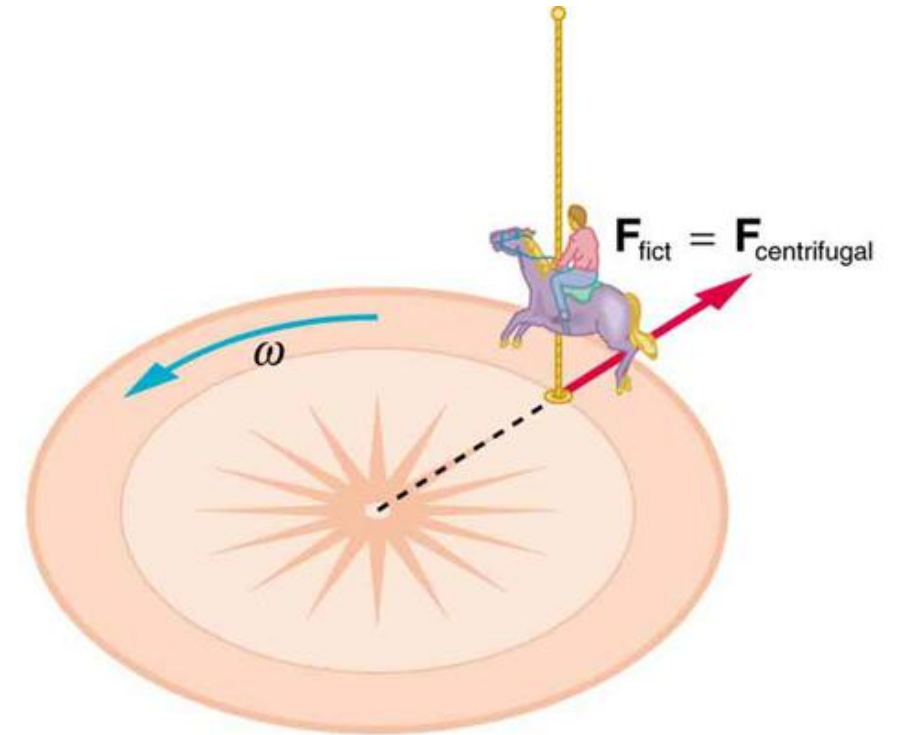
IN PHYSICS: AN EXAMPLE



Reference System is everything



V
S



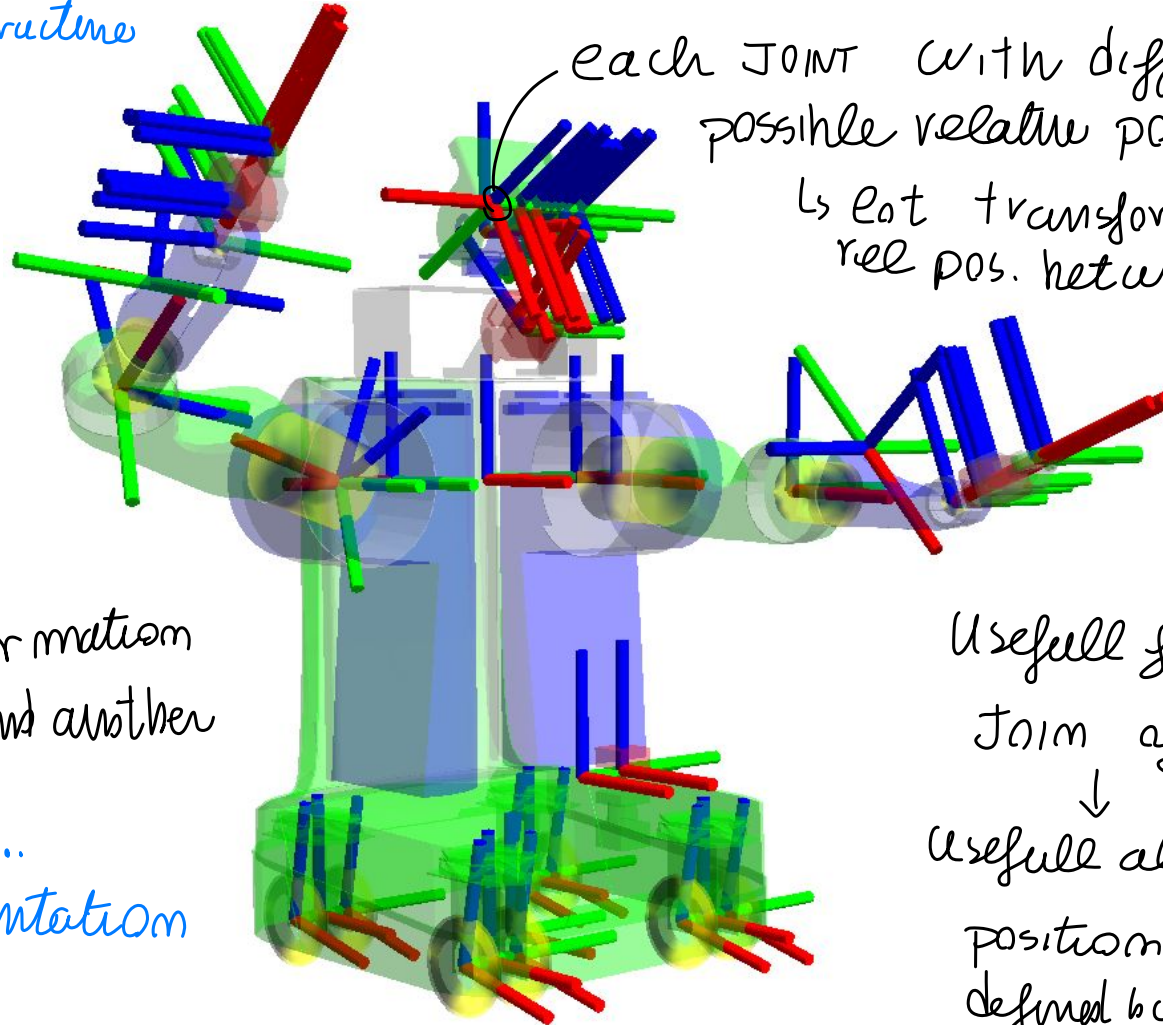
TF: powerful tool of ROS, to handle frame transformation

compute relative position of different components
of your environment ROBOTS

IN ROBOTICS



usually this is the structure
of your ROBOT $\Rightarrow$



each JOINT with different
possible relative position

$\hookrightarrow$ lot transform needed for
rel pos. between joints

ROS provide
tools for this
computation!

each joint defined as
static/dynamic transformation
between one joint and another

$\Downarrow$
relative position defined...
from that tree representation

Useful for Dynamic
Joint of Robots

Useful also for **ROBOT NAVIGATION**
position defined by joint

IN ROBOTICS



For manipulators:

- A moving reference frame for each joint
- A base reference frame
- A world reference frame

For autonomous vehicles:

- A fixed reference frame for each sensor
- A base reference frame
- A world reference frame
- A map reference frame

The frames are described in a tree and each frame comes with a transformation between itself and the father/child

The world frame is the most important, but the others are used for

FROM ONE FRAME TO ANOTHER



How is it possible to convert from a frame to another? *Math*, lot of it.

In a tree of reference frames:

Define a roto-translation between parent and child

Combine multiple roto-translation to go from the root to the



When the full transformation tree is available

Does all the hard work for us!

Interpolation, transformation, tracking

Keep track of all the dynamic transformation for a limited period of time

Decentralized

Provides position of a point in each possible reference frame

TF TREE TOOLS



ROS offers different tools to analyze the transformation tree:

```
-roslaunch rqt_tf_tree rqt_tf_tree
```

shows the tf tree at the current time

```
-roslaunch tf_view_frames
```

listen for 5 seconds to the /tf topic and create a pdf file with the tf tree



HOW TF_TREE SHOULD LOOK LIKE

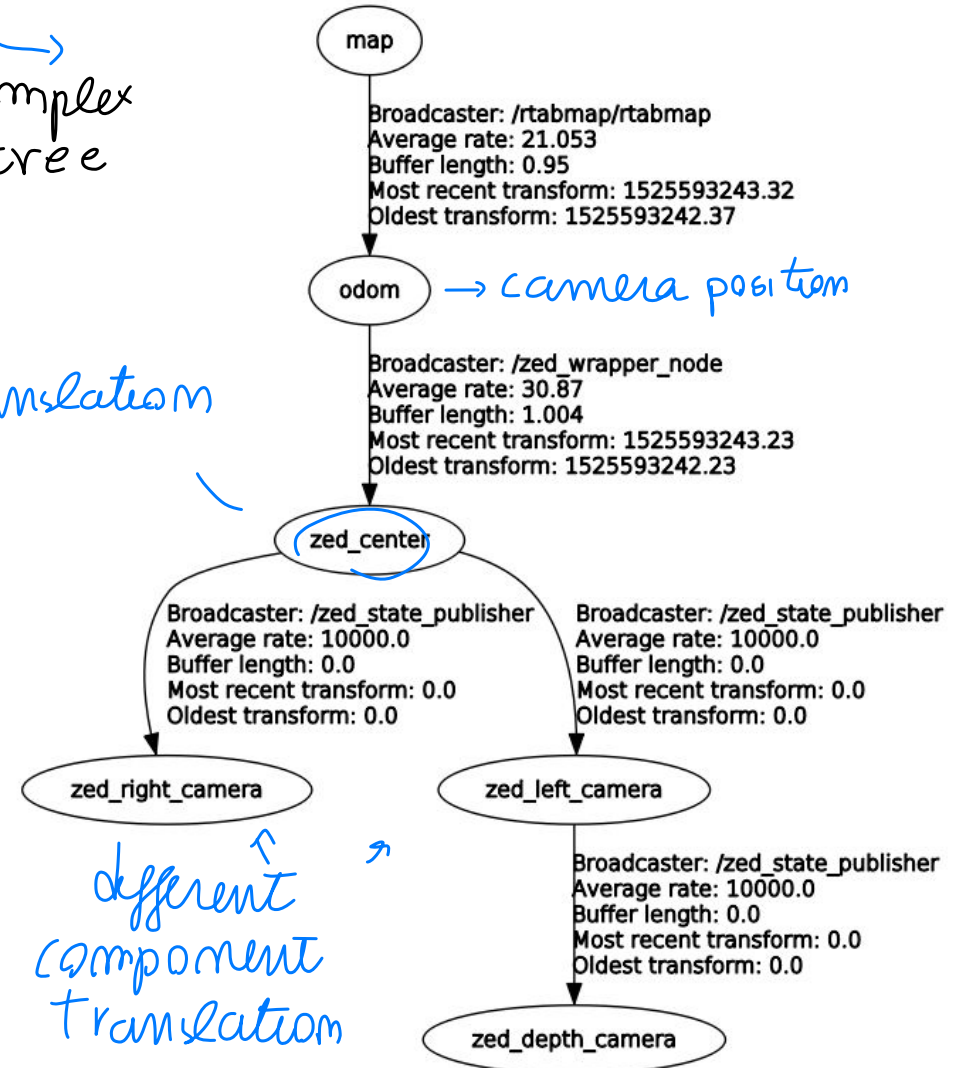
{ used for odometry } ← (example) stereocamera TF



move TF tree on simple sensor

complex TF tree

+ translation



~ this can be tested using Recordings!



robotics.bag (stereocamera recording to visualize TF3)

↳ how to use "bag" file...



• ".bag" is a standard way to record data inside ROS



to experiment on robot data, "bag" records all happening inside the robot on that time

↓
topics
timestamps → to check it later
...



(om.bag if recording custom messages, bad to transfer data)



how to play it:

» **rosbag play bag-name**

↳ you need roslaunch network! to play it

↳ start running give you data ongoing of "57s" of record
when you want to run in loop -- play **-l** bag-name

loop
play!

twice faster

-l (-r) 2

ratio of play → 1F Faster

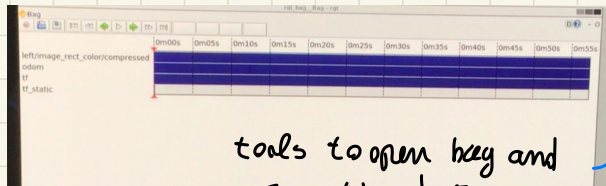
--clock

→ With this option, Rostime on bag file time, NOT on device time!
} necessary when you want timestamp on the bag file recordings!

also as

» **rqt-bag**

data visualization
of bag file



tools to open bag and
see the data

↑ topic...
+ fleshed
messages
on time

>> vashag info bag - name

↑

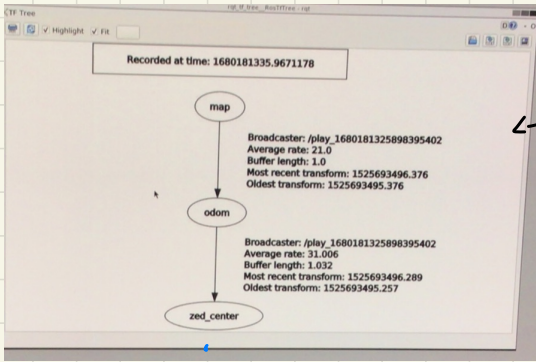
(look INFO inside the bag file!)

so we can play
the bag file and
look on its TF tree

>> rosbag play [-l] bag - name
↳ keep it running

>> rosrn rqt_tf-tree rqt_tf-tree

↓
rosmetwork tf-tree visualized!



topic currently
shown on TF tree

! more components →

having more component,
on ROS you have
STATIC / DYNAMIC TF

2 classes, because
some transform
are static while
other dym! ⇒ taking data

{ once you see STATIC TF... publish } → STATIC TF publish
{ only once @ beginning, } → once @ beginning
While dyn. TF → changing
during time ⇒ when restarting receive dym. data
refreshing

⇒ rqt-bag → open again we have

{ tf-static → only one message initially
tf-dynamic → publishing continuously

how to use and publish this TF?

↓
on /tf folder... /src .. pub.cpp

```
#include "ros/ros.h"
#include "turtlesim/Pose.h"
#include <tf/transform_broadcaster.h>

class tf_sub_pub{
public:
    tf_sub_pub(){
        sub = n.subscribe("/turtle1/pose", 1000, &tf_sub_pub::callback, this);
    }

    void callback(const turtlesim::Pose::ConstPtr& msg){
        tf::Transform transform; create transform
        transform.setOrigin( tf::Vector3(msg->x, msg->y, 0) ); ← take position
        tf::Quaternion q; ← set the origin of trans.
        q.setRPY(0, 0, msg->theta); ← 3D space, set (x,y,z) of position
    }
};
```

mode to receive data from turtle sim and publish back it as a TF message!

sub+pub Data on that mode

convert into standard ROS transformation which represent position+orientations of object

orientation as a quaternion (Roll Pitch yaw) set

position to publish

```
transform.setRotation(q); ← set orientation of trans.
br sendTransform(tf::StampedTransform(transform, ros::Time::now(), "world", "turtle1")); ← publish by broadcaster → pos, orient, time stamp → TF trans.
private:
    ros::NodeHandle n;
    tf::TransformBroadcaster br;
    ros::Subscriber sub;
};

int main(int argc, char **argv){
    ros::init(argc, argv, "subscribe_and_publish");
}
```

Time stamp! route ref frame: world

send Transform, all present

to publish a TF, we use tf::TransformBroadcaster object!

↳ "turtle")

↓
publisher transformation by route frame world and turtle frame

↓ TESTING...

» roscore

» roslaunch tf_examples tf-pub

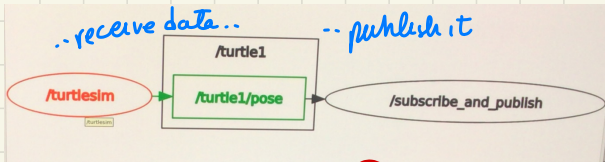
.. publish when turtle is running!

» roslaunch turtlesim turtlesim_mode

» rostopic list ~ /tf topic running all INFO by turtle position and time stamp...

» rqt-graph

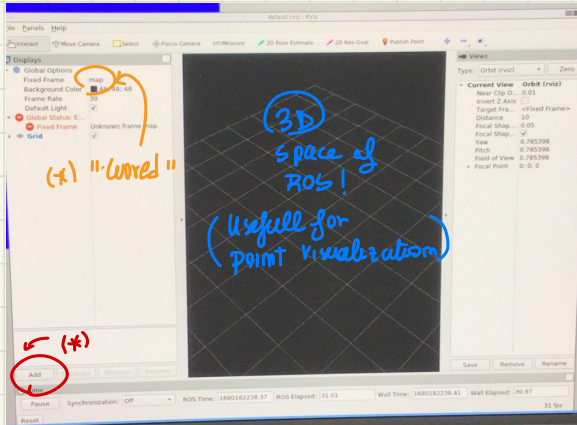
all as expected



convert position to TF to visualize it

from numerical value given by TF, we can use graphical tool to visualize the data !! → rviz

rosviz (standard visualization tool of ROS) 3D visualization



to see the TF position file

/Add (*)

on "By display type"
TF is a specific kind of topic

/TF

so you have TF
on workspace 3D

to visualize them

you have to change the Global options

from "map" → we are publishing

Transform between WORLD to TURTLE!

to connect turtle to world

write "world" → so you see turtle position
respect the REF frame!

⇓

and if moving turtle by teleop-key you
see it moving in the world frame!

{ received position broadcast
publishing TF on the network }

TF changing dummy time (dym)

+ TF static dummy time → when "static" they don't change over
time → so you don't need a broadcaster!

⇓

instead you use static-tf-publisher!

STATIC TF

tf published using launch files

by standard ROS mode that can be called by cmdline (complex)

OR usually write static TFom launch file

+turtle.launch

```
<?xml version="1.0" encoding="UTF-8" ?>
<launch>
  <node pkg="tf" type="tf_pub" name="tf_pub"/>
  <node pkg="turtlesim" type="turtlesim_node" name="turtlesim_node"/>
  <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key"/>
  <node pkg="tf" type="static_transform_publisher" name="back_right" args="0.3 0.3 0.3 0 0 0 1" />
  <node pkg="tf" type="static_transform_publisher" name="front_right" args="0.3 0.3 0.3 0 0 0 1" />
  <node pkg="tf" type="static_transform_publisher" name="front_left" args="0.3 0.3 0.3 0 0 0 1" />
  <node pkg="tf" type="static_transform_publisher" name="back_left" args="0.3 0.3 0.3 0 0 0 1" />
</launch>
```

to start the mode to use

→ to publish a static tf

you use tf static-transform-publ.

↓
you are publishing different
name... specify:

"x y z Quaternion route-frame frequency-publishing"

↑ static TF publishing procedure!

With new tf version → TF publisher better to run, easier!

"pos orient route-frame child-frame freq" ← how to publish TF in ROS!
easy to define → so on launch file!

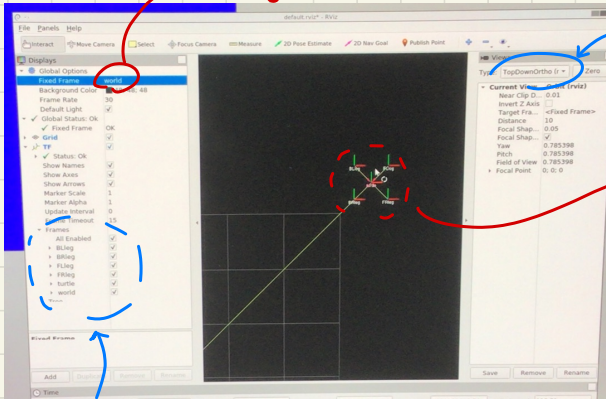
» roslaunch tf-example turtle.launch

↓
It starts all ... so on (RVIZ) initializing 'TF' and 'world'

{ We see all
the frames of our ROBOT to select which one to show
(the useful ones!)
(on left menu)

{ Visualization } → on top right option
option

Route of the TF tree is the better Reference $\rightarrow$ but you can't see the POV as you wish (in the middle!)



visualization

Turtle with all different frames visible respect its reference!

frames to see

WRITE THE TF PUBLISHER



Now that we got an idea regarding how tf works and why it's useful we can take a look on how to write a tf broadcaster

Usually to do this you need a robot,
we could still use a bag publishing odometry,
but turtlesim is still a good option.

WRITE THE TF PUBLISHER



Subscribe to `/turtlesim/pose`

convert the pose to a transformation

publish the transformation referred to a world frame

add 4 static transformation for the 4 turtle's legs



WRITE THE TF PUBLISHER

Create a package called `tf_turtlebot` inside you catkin environment adding the `roscpp`, `std_msgs` and `tf` dependencies:

```
$ catkin_create_pkg tf_turtlebot std_msgs roscpp tf
```

now cd to the package src folder and create the file `tf_publisher`

```
$ gedit tf_publisher.cpp
```

WRITE THE TF PUBLISHER



First we write some standard include:

```
#include "ros/ros.h"
```

```
#include "turtlesim/Pose.h"
```

```
#include <tf/transform_broadcaster.h>
```

WRITE THE TF PUBLISHER



Then we write the main function:

```
int main(int argc, char **argv)
{
    ros::init(argc, argv, "subscribe_and_publish");
    tf_sub_pub my_tf_sub_bub;
    ros::spin();
    return 0;
}
```

Notice that we still have to initialize ros, but we are not creating the node handle here, instead we instantiate an object of class tf_sub_pub

WRITE THE TF PUBLISHER



Now we have to create our class:

```
class tf_sub_pub
{
    public:
    tf_sub_pub(){
    }
    private:

};
```

WRITE THE TF PUBLISHER



First we declare as private the node handle:

```
ros::NodeHandle n;
```

Then we create the subscriber and the tf broadcaster:

```
tf::TransformBroadcaster br;  
ros::Subscriber sub;
```

WRITE THE TF PUBLISHER



Now we can call the subscribe function inside the class constructor:

```
sub = n.subscribe("/turtle1/pose", 1000, &tf_sub_pub::callback, this);
```

Then we write the callback function:

```
void callback(const turtlesim::Pose::ConstPtr& msg){  
}
```

WRITE THE TF PUBLISHER



Inside the callback we create a transform object:

```
tf::Transform transform;
```

and populate it using the data from the message (we are in a 2D environment):

```
transform.setOrigin( tf::Vector3(msg->x, msg->y, 0) );  
tf::Quaternion q;  
q.setRPY(0, 0, msg->theta);  
transform.setRotation(q);
```

WRITE THE TF PUBLISHER



Last we publish the transformation using the broadcaster; the stampedtransform function allow us to create a stamped transformation adding the timestamp, our custom transformation, the root frame and the child frame:

```
br.sendTransform(tf::StampedTransform(transform, ros::Time::now(), "world", "turtle"));
```


THE CODE



```
#include "ros/ros.h"
#include "turtlesim/Pose.h"
#include <tf/transform_broadcaster.h>
class tf_sub_pub
{
public:
    tf_sub_pub(){
        sub = n.subscribe("/turtle1/pose", 1000, &tf_sub_pub::callback, this);
    }
    void callback(const turtlesim::Pose::ConstPtr& msg){
        tf::Transform transform;
        transform.setOrigin( tf::Vector3(msg->x, msg->y, 0) );
        tf::Quaternion q;
        q.setRPY(0, 0, msg->theta);
        transform.setRotation(q);
        br.sendTransform(tf::StampedTransform(transform, ros::Time::now(), "world", "turtle"));
    }
private:
    ros::NodeHandle n;
    tf::TransformBroadcaster br;
    ros::Subscriber sub;
};
int main(int argc, char **argv)
{
    ros::init(argc, argv, "subscribe_and_publish");
    tf_sub_pub my_tf_sub_bub;
    ros::spin();
    return 0;
}
```



WRITE THE TF PUBLISHER

Now as usual we have to add this new file to the CMakeLists file. We specified the dependencies during the package creation, so we only need to add the lines:

```
add_executable(tf_turtlebot
  src/tf_publisher.cpp
)
add_dependencies(tf_turtlebot ${${PROJECT_NAME}_EXPORTED_TARGETS}
  ${catkin_EXPORTED_TARGETS})
target_link_libraries(tf_turtlebot
  ${catkin_LIBRARIES}
)
```

TESTING



Now we can cd to the root of the environment and compile everything

Before adding the legs transformation we can test our code:

run turtlesim, turtlesim_teleop and our node, then open rviz to visualize the tf

```
$ roscore
```

```
$ rosrun turtlesim turtlesim_node
```

```
$ rosrun turtlesim turtle_teleop_key
```

```
$ rosrun tf_turtlebot tf_turtlebot
```

```
$ rviz
```

ADD STATIC TF



After properly testing our code we can add the other tf.

But the legs tf are fixed from the turtlebot body, so we don't need to write a tf broadcaster like we did, we can simply run them using the static transform node

We don't want to manually start four tf in four different terminals, so we will create a launch file:

create a folder launch and a file called launch.launch

ADD STATIC TF



The launch file will have as usual the `<launch>` tags and the node we previously wrote:

```
<launch>  
<node pkg="tf_turtlebot" type = "tf_turtlebot" name = "tf_turtlebot"/>  
</launch>
```

We can also add the two turtlesim node:

```
<node pkg="turtlesim" type = "turtlesim_node" name = "turtlesim_node"/>  
<node pkg="turtlesim" type = "turtle_teleop_key" name = "turtle_teleop_key"/>
```

ADD STATIC TF



Now we will add the four static tf specifying in the args field the position (x,y,z) and the rotation as a quaternion (qx,qy,qz,qw) then the root frame, the child frame and the update rate:

```
<node pkg="tf" type="static_transform_publisher" name="back_right" args="0.3 -0.3 0 0 0 0 1 turtle FRleg 100" />
<node pkg="tf" type="static_transform_publisher" name="front_right" args="0.3 0.3 0 0 0 0 1 turtle FLleg 100" />
<node pkg="tf" type="static_transform_publisher" name="front_left" args="-0.3 0.3 0 0 0 0 1 turtle BLleg 100" />
<node pkg="tf" type="static_transform_publisher" name="back_left" args="-0.3 -0.3 0 0 0 0 1 turtle BRleg 100" />
```

Now we will only need to call the launch file to start all the nodes:

```
$ roslaunch tf_turtlebot launch.launch
```

ADD STATIC TF



Now run `rqt_tf_tree` to show the tf tree and `rviz` for the visual representation of the turtle position

If you want to see the published tf you can use `rostopic echo`, but also:

```
$ rosrun tf tf_echo father child
```

```
$ rosrun tf tf_echo \world \FRleg
```

ACTIONLIB

ROBOTICS



POLITECNICO
MILANO 1863

WHAT IS ACTIONLIB



Node A sends a request to node B to perform some task

Service

Small execution time

Requesting node can wait

No status

No cancellation

Action

Long execution time

Requesting node cannot wait

Status monitoring

Cancellation

WHAT IS ACTIONLIB



actionlib package is:

- sort of ROS implementation of threads

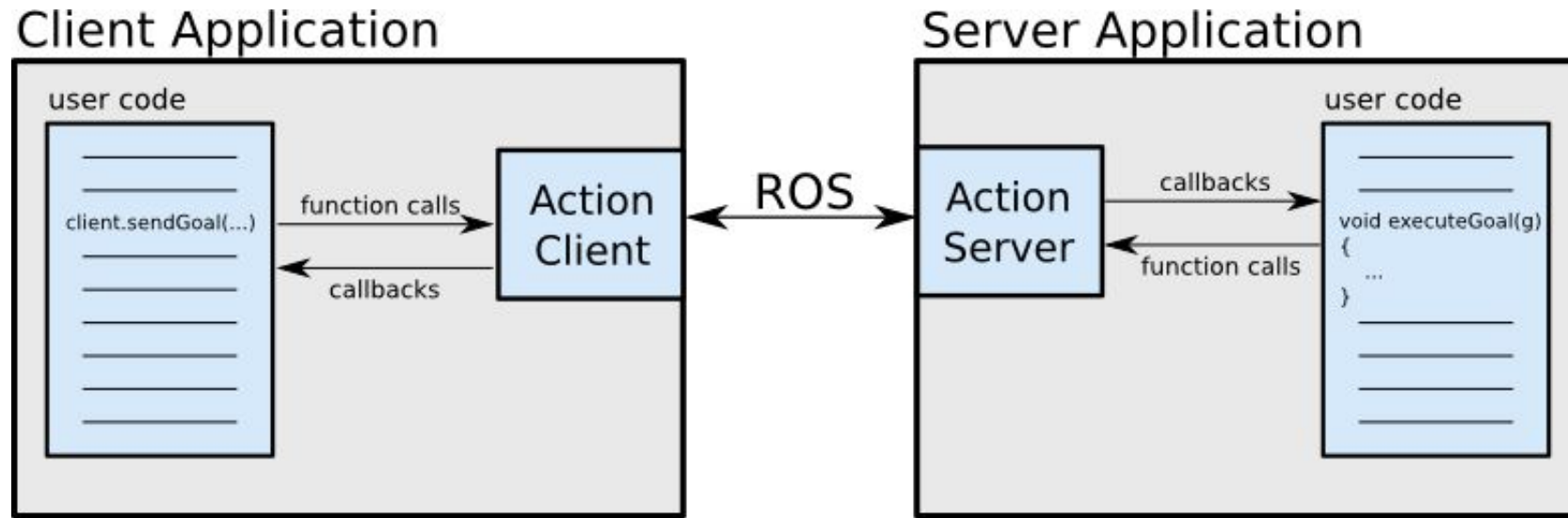
- based on a client/server paradigm

And provides tools to:

- create servers that execute long-running tasks (that can be preempted).

- create clients that interact with servers

WHAT IS ACTIONLIB

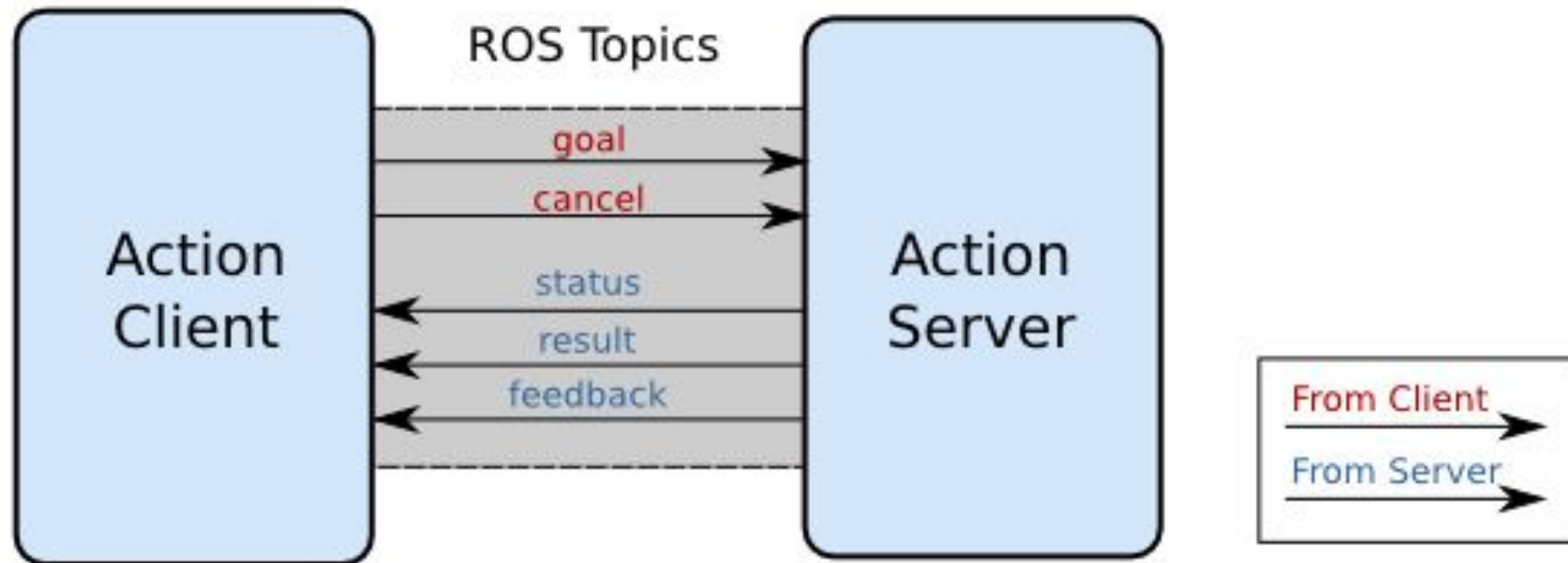


The ActionClient and ActionServer communicate via a "ROS Action Protocol", which is built on top of ROS messages

CLIENT-SERVER INTERACTION



Action Interface



CLIENT-SERVER INTERACTION



goal: to send new goals to server

cancel: to send cancel requests to server

status: to notify clients on the current state of every goal in the system.

feedback: to send clients periodic auxiliary information for a goal

result: to send clients one-time auxiliary information upon completion of a goal

ACTION AND GOAL ID

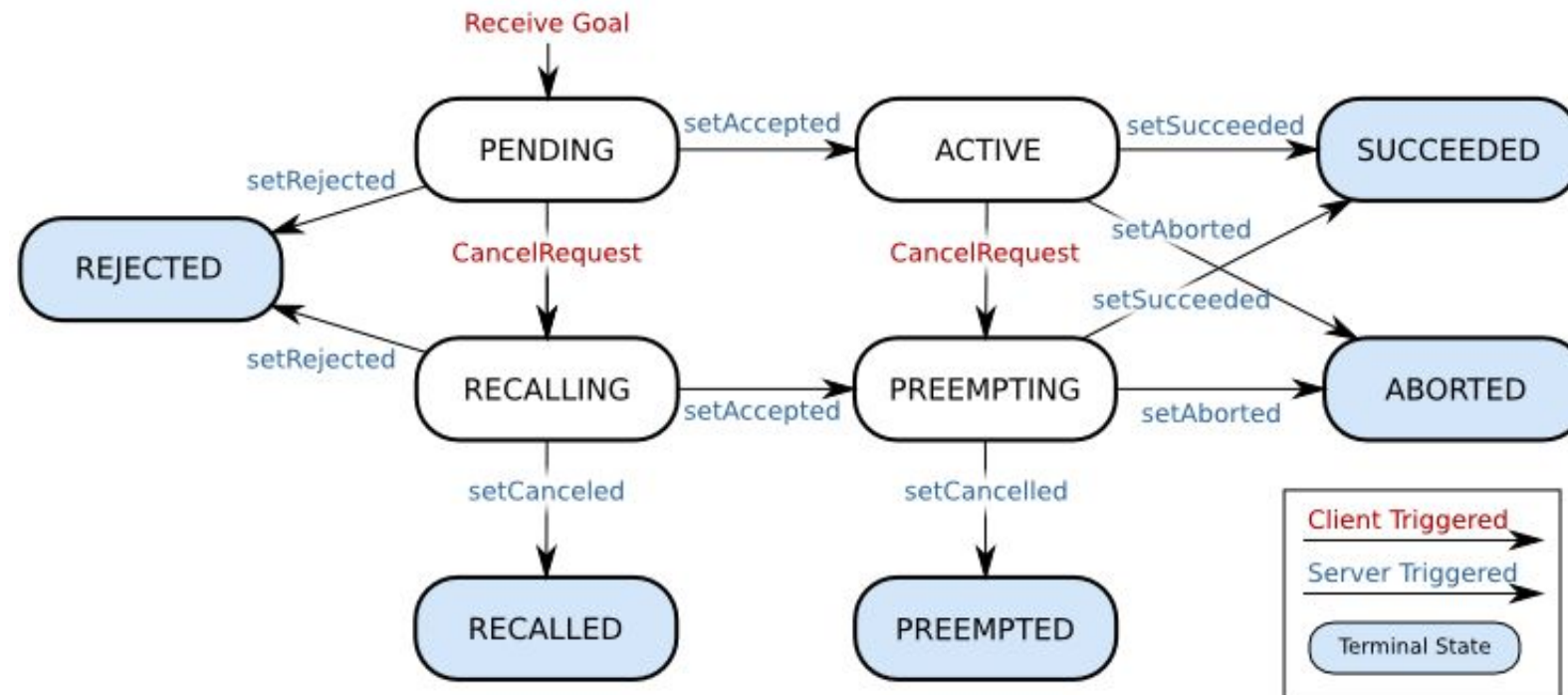


Action templates are defined by a name and some additional properties through an `.action` structure defined in ROS

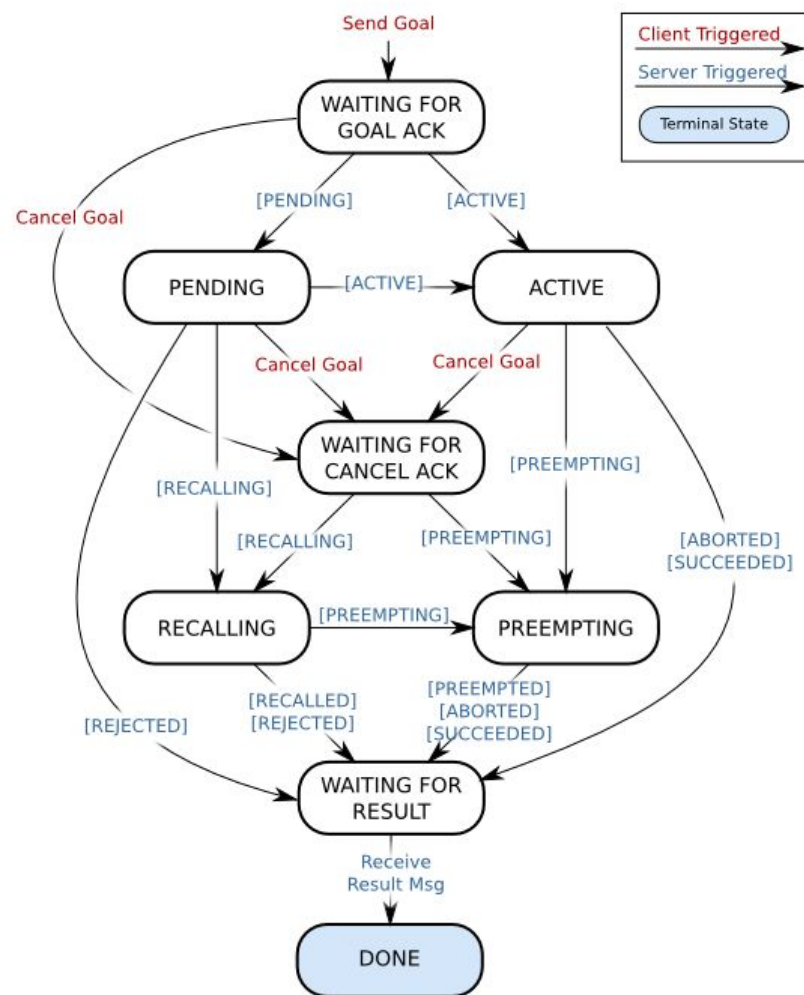
Each *instance* of an action has a unique Goal ID

Goal ID provides the action server and the action client with a robust way to monitor the execution of a particular instance of an action.

SERVER STATE MACHINE



CLIENT STATE MACHINE



.ACTION EXAMPLE



Define the goal

uint32 dishwasher_id # Specify the dishwasher id

Define the result

uint32 total_dishes_cleaned

Define a feedback message

float32 percent_complete

SIMPLEACTIONSERVER



```
int main(int argc, char** argv) {  
    ros::init(argc, argv, "do_dishes_server");  
    ros::NodeHandle n;  
    Server server(n, "do_dishes", boost::bind(&exe, _1, &server), false);  
    server.start();  
    ros::spin();  
    return 0;  
}
```

SIMPLEACTIONSERVER



```
void exe(const chores::DoDishesGoalConstPtr& goal, Server* as) {
    while(allClean()) {
        doDishes(goal->dishwasher_id)
        if(as->isPreemptRequested() || !ros::ok()) {
            as->setPreempted();
            break;
        }
        as->publishFeedback(currentWork(goal->dishwasher_id))
    }
    if(currentWork(goal->dishwasher_id) == 100)
        as->setSucceeded();
}
```

SIMPLEACTIONCLIENT



```
#include <chores/DoDishesAction.h>
```

```
#include <actionlib/client/simple_action_client.h>
```

```
typedef actionlib::SimpleActionClient<chores::DoDishesAction> Client;
```

SIMPLEACTIONCLIENT



```
int main(int argc, char** argv) {  
    ros::init(argc, argv, "do_dishes_client");  
    Client client("do_dishes", true); // true -> don't need ros::spin()  
    client.waitForServer();  
    chores::DoDishesGoal goal;  
    //set goal parameters  
    goal.dishwasher_id = pickDishwasher();  
}
```

SIMPLEACTIONCLIENT



```
client.sendGoal(goal);
client.waitForResult(ros::Duration(5.0));
if (client.getState() == actionlib::SimpleClientGoalState::SUCCEEDED)
    ROS_INFO("Yay! The dishes are now clean");
std::string state = client.getState().toString();
ROS_INFO("Current State: %s\n", state.c_str());
return 0;
}
```

TESTING



Copy the `actionlib_tutorial` folder inside the `src` folder of your catkin workspace and compile it

To start the server:

```
$ rosrun actionlib_tutorials fibonacci_server
```

The client has some parameters that can be set in the launch file, order and duration; after setting those parameters call:

```
$ roslaunch actionlib_tutorials launcher.launch
```

TESTING



You can monitor the server status simply using topics:

```
$ rostopic list
```

To get the feedback from the server:

```
$ rostopic echo /fibonacci/feedback
```


MESSAGE FILTERS

ROBOTICS



POLITECNICO
MILANO 1863

MESSAGE FILTERS



Useful to synchronize multiple topics

Need topics with header and timestamp

Can synchronize with exact time or approximate time

Camera topics synchronization has a custom version



MESSAGE FILTERS (without policy)

```
message_filters::Subscriber<geometry_msgs::Vector3Stamped> sub1(n, "topic1",  
1);
```

← **Create the subscriber**

```
message_filters::Subscriber<geometry_msgs::Vector3Stamped> sub2(n, "topic2",  
1);
```

```
message_filters::TimeSynchronizer<geometry_msgs::Vector3Stamped,  
geometry_msgs::Vector3Stamped> sync(sub1, sub2, 10);
```

```
sync.registerCallback(boost::bind(&callback, _1, _2));
```

↑
Bind it with the callback

↑
Create the time synchronizer



MESSAGE FILTERS (with policy)

```
typedef  
message_filters::sync_policies::ExactTime<geometry_msgs::Vector3Stamped,  
geometry_msgs::Vector3Stamped> MySyncPolicy;
```



Create the policy

```
message_filters::Synchronizer<MySyncPolicy> sync(MySyncPolicy(10), sub1, sub2);  
sync.registerCallback(boost::bind(&callback, _1, _2));
```



Bind it with the callback



Create the time synchronizer with
the policy